Assignment Dossier: Operation Cyber-Histology

`Post-Incident Engineering and Pipeline Reconstruction`

RESTRICTED // FOR SYSTEM ARCHITECTS ONLY
To: Freelance Machine Learning Expert
From: Incident Response Team, BioHealth Diagnostics Global
Subject: Total Pipeline Failure Following Malicious Corporate Espionage
Urgency: CRITICAL / IMMEDIATE ACTION REQUIRED

Incident Report: Twenty-four hours ago, BioHealth Diagnostics was on the verge of deploying its next-generation multi-class clinical triage pipeline. Our core model registry, **AlexNet, VGG16, and ResNet18**, was calibrated to run seamlessly across our standardized diagnostic image profiles (`cells, chest, lesions` and `orgs`).

Before fleeing the campus to evade security, disgruntled former architect Dr. Viktor Vance executed an environment-wipe utility. The damage is extensive:

- All trained weights, production-ready source code, configuration registries, and evaluation automation scripts were permanently deleted.
- Forensics managed to scrape the local drive caches and recovered older, heavily unoptimized, and volatile draft copies of `data.py`, `models.py`, `train.py`, and `trainer.py`.
- All configuration assets (`config.json`), automated testing frameworks, and multi-dataset comparison tools were completely unrecoverable.

Preliminary execution attempts on the recovered code show that it is highly unstable. It suffers from a mix of immediate runtime exceptions, silent computational graph corruptions, numerical gradient explosions, and algorithmic anti-patterns.

Requested Action: Audit the volatile remains of the codebase, neutralize every mathematical and syntactic landmine left behind in the scripts, forge the missing configuration infrastructure from scratch, and deploy an automated multi-dataset benchmark runner to restore project integrity before the system completely goes dark. The incident response team expects the below listed deliverables as commits to a dedicated branch named `en_xxxx`, where xxxx is your student enrollment number, in the project repository `https://git.fiw.fhws.de/teaching/mai/idl26_finalassignment`, with clear commit messages indicating the nature of the changes made. **Deadline: 9.7.2026, midnight German time**.

1. **Complete Corrected Codebase:** A fully functional, production-grade version of the entire repository, including `data.py`, `models.py`, `train.py`, and `trainer.py`, alongside your newly engineered configuration and testing frameworks. The codebase must be modular, error-free, compatible with all four target datasets, and capable of executing training and predictions for all three models via native Python calls driven by your configuration structure.
2. **Incident Audit Log:** A technical table (submitted as `AUDIT_LOG.md`) itemizing every bug, corruption, or anti-pattern discovered within the recovered code. For each entry, you must explicitly document: (i) the file name, (ii) how the problem manifests, (iii) the mathematical or logical root cause of the failure, (iv) the structural correction implemented, and (v) the exact git commit hash containing the fix.
3. **Consolidated Benchmark Report:** A brief evaluation report (submitted as `REPORT.md` or `REPORT.pdf` in your repository root) comparing the final performance of the corrected models across all configuration permutations. This must include a clean summary table capturing key metrics (accuracy, precision, recall, and macro F1-score) along with ar-

chitectural recommendations for optimal dataset/model pairings based on your observed benchmarks.

System State and Engineering Requirements

The forensic team has been able to observe some patterns in the types of failures that are occurring when attempting to run the recovered code. So far, they have identified the following categories of issues that are plaguing the system and could be a useful starting point for your audit and reconstruction efforts:

Crashing code / runtime errors: Bugs that cause Python or PyTorch to throw an error and halt execution, e.g., syntax typos, shape mismatches, and incorrect data types.

Silent logical flaws: Hidden logical flaws where the scripts run smoothly without crashing, but the underlying math or logic is completely invalid, e.g., broken data boundaries, hidden data leaks, or shadowed variables that silently break the pipeline.

Numerical and gradient failures: Situations where models flatline or explode into useless numbers. You need to find out why the mathematical signals inside the network are getting trapped, neutralized, or blown out of proportion during execution.

Rigid infrastructure: Rigid, inflexible parts of the code that must be cleaned up into dynamic, modular variables so that the entire training and testing pipeline can be controlled cleanly through a single external configuration file.

RESTRICTED // FOR SYSTEM ARCHITECTS ONLY
To: Freelance Machine Learning Expert
From: Executive Board BioHealth Diagnostics Global
Subject: Environmental Impact - Green Initiative
Urgency: CRITICAL / IMMEDIATE ACTION REQUIRED

Issue: The
